Java Interview Preparation

Part 1: Core Java, Collections, JPA/Hibernate, Spring Boot & SQL

Section 1: OOPs, Core Java & JVM Architecture (Q1–Q20)

Q1: What happens under the hood when a class is loaded by the JVM ClassLoader?

ANSWER

Class loading involves 3 main phases:

Loading: Reads binary .class bytecodes and creates a java.lang.Class object in the Metaspace/Heap. Follows Delegation Hierarchy: Bootstrap ClassLoader → Platform ClassLoader → Application ClassLoader.

Linking:

Verification: Validates bytecode against JVM rules (type safety, stack overflow checks).

Preparation: Allocates memory for static variables and initializes them to default values (e.g., 0, null).

Resolution: Replaces symbolic references in the constant pool with direct memory references.

Initialization: Executes static initializers and assigns explicitly defined values to static fields (static { ... }).

Q2: What is Java Metaspace, and how does it differ from the old PermGen?

ANSWER

Introduced in Java 8, Metaspace replaced PermGen for storing class metadata, method data, and constant pools.

PermGen: Was located in contiguous Heap memory with a fixed maximum size (-XX:MaxPermSize), leading to frequent java.lang.OutOfMemoryError: PermGen space.

Metaspace: Uses native off-heap memory. It grows dynamically by default (bounded only by system RAM) and reduces OOM risks. It can be capped using -XX:MaxMetaspaceSize.

Q3: Explain the difference between Deep Copy and Shallow Copy in Java.

ANSWER

Shallow Copy: Copies primitive attributes as values and object references as memory addresses. Both original and cloned objects share references to the same internal nested objects.

Deep Copy: Recursively copies all field values and creates new instances of nested objects, making the cloned instance completely independent.

```
// Shallow Copy using Object.clone()
class Address { String city; }
class Person implements Cloneable {
    Address address;
    protected Object clone() throws CloneNotSupportedException {
        return super.clone(); // Shallow: shares address reference
    }
}
```

Q4: Why is String immutable in Java, and how does it affect security and memory?

ANSWER

String Pooling (Memory): Saves heap memory by sharing duplicate string literals in the String Constant Pool (SCP).

Security: Strings carry sensitive parameters (database URLs, user credentials, network sockets). Immutability prevents external alteration after validation.

Thread Safety: Read-only access allows multiple threads to access string instances without synchronization.

Hashcode Caching: hashCode() is calculated once upon creation and cached, making Strings optimal map keys.

Q5: Explain the contract between equals() and hashCode(). What breaks if violated?

ANSWER

Contract:

If a.equals(b) is true, a.hashCode() must equal b.hashCode().

If a.hashCode() == b.hashCode(), a.equals(b) may or may not be true (hash collision).

Overriding equals() requires overriding hashCode().

Consequence of Violation: Storing objects in hash-based collections (HashMap, HashSet) breaks. If equals() is overridden without hashCode(), equal objects will generate different hashes and be stored in different buckets, leading to duplicate entries or failed lookups (get() returns null).

Q6: How does Method Overriding work with Exception Handling rules?

ANSWER

Checked Exceptions: The overriding method in a subclass cannot declare new or broader checked exceptions than those declared by the superclass method. It can declare narrower exceptions, fewer exceptions, or none.

Unchecked Exceptions: No restrictions (can throw any RuntimeException).

Q7: Differentiate between String, StringBuilder, and StringBuffer.

ANSWER

String: Immutable, thread-safe, slower for repeated modifications.

StringBuilder: Mutable, not thread-safe, faster execution speed (use inside local method contexts).

StringBuffer: Mutable, thread-safe (all methods synchronized), slower than StringBuilder.

Q8: What are Object-Level Locks vs. Class-Level Locks in Java?

ANSWER

Object-Level Lock: Synchronizes instance methods (synchronized void method()) or blocks (synchronized(this)). Controls thread access to instance-specific data across distinct object instances.

Class-Level Lock: Synchronizes static methods (synchronized static void method()) or class objects (synchronized(MyClass.class)). Prevents multiple threads from entering static synchronized code blocks across all instances of that class.

Q9: What is the purpose of System.gc() and Runtime.getRuntime().gc()?

ANSWER

Both issue non-binding requests to the JVM asking it to run Garbage Collection. The JVM is not obligated to execute GC immediately upon invocation. Calling System.gc() manually is generally discouraged in production because it triggers full GC sweeps, pausing application threads (Stop-The-World).

Q10: What are Phantom, Weak, and Soft References in Java?

ANSWER

Soft Reference (SoftReference<T>): Garbage collected only when the JVM runs low on memory. Ideal for memory-sensitive caches.

Weak Reference (WeakReference<T>): Garbage collected during the next GC cycle regardless of available memory if no strong references exist. Used in WeakHashMap.

Phantom Reference (PhantomReference<T>): Collected after target object finalization. Used with ReferenceQueue to perform custom low-level resource cleanup as a safe alternative to finalize().

Q11: Explain abstract classes vs. interfaces (Java 8+ context).

ANSWER

Abstract Class: Can maintain state (instance fields), constructors, non-final instance variables, and support single inheritance.

Interface (Java 8+): Can contain default and static methods (and private methods from Java 9). Interfaces cannot hold instance state/fields (only public static final constants) and support multiple inheritance.

Q12: How does try-with-resources work, and what is AutoCloseable?

ANSWER

Introduced in Java 7, try-with-resources automatically closes managed resources when exiting the block. Resources declared must implement java.lang.AutoCloseable or java.io.Closeable.

```
try (BufferedReader reader = new BufferedReader(new FileReader("file.txt"))) {
    return reader.readLine();
} // reader.close() called implicitly in reverse creation order
```

Q13: What are Suppressed Exceptions in Java?

ANSWER

Occur when an exception is thrown inside a try block and another exception is thrown while automatically closing a resource in try-with-resources. The try-block exception takes precedence, and the close exception is added to its suppressed list (ex.getSuppressed()).

Q14: Explain the final, finally, and finalize keywords.

ANSWER

final: Modifier used to make variables unmodifiable, methods un-overridable, and classes un-extendable.

finally: Block following try-catch executed regardless of whether exceptions were thrown or caught.

finalize: Method defined on Object called prior to garbage collection (deprecated since Java 9).

Q15: What is marker interface and name standard ones in Java?

ANSWER

An interface with no declared methods or fields. Serves as metadata marking a class for runtime processing (e.g., Serializable, Cloneable, RandomAccess).

Q16: What is Covariant Return Type in Java?

ANSWER

Subclasses overriding a parent class method can declare a return type that is a subclass of the superclass method's return type.

```
class Parent { Parent getSelf() { return this; } }
class Child extends Parent {
    @Override
    Child getSelf() { return this; } // Covariant return
}
```

Q17: What are composition and aggregation, and why favor composition over inheritance?

ANSWER

Aggregation: Weak HAS-A relationship where child objects can exist independently of parent (e.g., Department and Teacher).

Composition: Strong HAS-A relationship where child lifecycles depend entirely on the parent (e.g., Car and Engine).

Favoring Composition: Avoids fragile base class issues, tight coupling, and hierarchy rigidness associated with extends.

Q18: What is the purpose of volatile keyword?

ANSWER

Guarantees visibility of variable changes across multiple threads by reading/writing values directly to main memory rather than CPU L1/L2 caches. It does not guarantee atomicity for composite operations like i++.

Q19: Explain the Java Memory Model (JMM) Heap layout (G1 GC context).

ANSWER

The Java Heap is split into:

Young Generation: Divided into Eden and two Survivor spaces (S0/S1). New objects start in Eden.

Old (Tenured) Generation: Long-surviving objects moved here after threshold GC cycles.

G1 GC: Divides heap into non-contiguous equal-sized region blocks dynamically acting as Eden, Survivor, or Old spaces.

Q20: What causes StackOverflowError vs. OutOfMemoryError?

ANSWER

StackOverflowError: Thread execution stack exceeds memory limits due to deep/infinite recursion.

OutOfMemoryError (Java Heap Space): Heap space cannot allocate memory for new objects despite Garbage Collection runs.

Section 2: Multithreading & Concurrency (Q21–Q40)

Q21: What is the difference between Runnable and Callable?

ANSWER

Runnable: run() method returns void and cannot throw checked exceptions.

Callable<V>: call() method returns a result of generic type V and can throw checked exceptions.

Q22: Explain Thread States in Java lifecycle.

ANSWER

NEW: Thread created, not yet started (start() not called).

RUNNABLE: Thread executing or eligible for execution in OS pool.

BLOCKED: Waiting to acquire a monitor lock.

WAITING: Waiting indefinitely for another thread's signal (wait(), join()).

TIMED_WAITING: Waiting for specified duration (sleep(ms), wait(ms)).

TERMINATED: Execution completed or thrown unhandled exception.

Q23: How does ConcurrentHashMap achieve internal thread safety?

ANSWER

Prior to Java 8, it used Segment Locks (ReentrantLocks on array buckets).

In Java 8+, it uses a bucket array with Compare-And-Swap (CAS) instructions for inserting new nodes, and synchronized locks locked exclusively on the first head node of individual hash bins during updates, providing high concurrent write throughput.

Q24: What is the distinction between wait() and sleep()?

ANSWER

wait(): Defined in Object class. Releases monitor lock, must be called inside synchronized block/method, resumes on notify()/notifyAll().

sleep(): Defined in Thread class. Holds monitor lock, pauses execution for defined time, resumes automatically.

Q25: Explain Double-Checked Locking in Singleton design pattern.

ANSWER

Optimizes thread-safe singletons by eliminating synchronization overhead on every access. Requires volatile instance field to prevent instruction reordering during instantiation.

```
public class Singleton {
    private static volatile Singleton instance;
    public static Singleton getInstance() {
        if (instance == null) { // 1st check (unlocked)
            synchronized (Singleton.class) {
                if (instance == null) { // 2nd check (locked)
                    instance = new Singleton();
                }
            }
        }
        return instance;
    }
}
```

Q26: What is ThreadLocal and where is it used?

ANSWER

Provides thread-isolated variable values. Each accessing thread keeps an independent copy of the variable inside its internal ThreadLocalMap.

Use Case: Passing contextual metrics like user security tokens, database transaction contexts, or SimpleDateFormat across application layers without passing arguments.

Memory Leak Warning: ThreadLocal keys are WeakReferences, but values are strong references. Unremoved values in reused thread pools cause memory leaks. Always call ThreadLocal.remove() in finally blocks.

Q27: How does CompletableFuture differ from standard Future?

ANSWER

Standard Future operations are blocking (get()). CompletableFuture allows non-blocking asynchronous pipeline chaining (thenApply, thenCompose, thenCombine), exception handling wrappers (exceptionally), and explicit manual completion.

Q28: Explain CountDownLatch vs. CyclicBarrier.

ANSWER

CountDownLatch: Single-use barrier. Threads wait until a counter reaches zero via countDown(). Main thread waits for N worker threads.

CyclicBarrier: Reusable barrier. N worker threads wait for each other to reach a synchronization checkpoint before proceeding together (await()).

Q29: What is the Producer-Consumer Problem and how to resolve it using BlockingQueue?

ANSWER

Producer threads put data into a bounded buffer, while consumer threads process data from it. ArrayBlockingQueue or LinkedBlockingQueue manages concurrent synchronization natively via internal locks (ReentrantLock, Condition), blocking producers when full and consumers when empty.

Q30: What is False Sharing in Concurrency?

ANSWER

Occurs when dynamic variables modified by independent CPU cores reside on the same L1/L2 cache line (typically 64 bytes). Modifying one variable invalidates the entire cache line for other cores, causing performance degradation. Fixed using @Contended annotation or padding fields.

Q31: What are Executors and what are the standard thread pool types in Executors factory?

ANSWER

Abstraction layer separating task submission from thread management.

newFixedThreadPool(int n): Fixed number of worker threads, unbounded queue.

newCachedThreadPool(): Dynamic thread count, reuses idle threads, creates new ones as needed.

newSingleThreadExecutor(): Single worker thread sequentially running tasks.

newScheduledThreadPool(int core): Schedules commands to run periodically or after delays.

Q32: What is Thread Starvation vs. Deadlock vs. Livelock?

ANSWER

Deadlock: Two or more threads are permanently blocked, waiting on locks held by each other.

Livelock: Threads continuously change states in response to each other without making actual forward progress.

Starvation: A thread is perpetually denied CPU time or resource access due to greedy higher-priority threads.

Q33: How do you detect and fix Deadlocks in production environments?

ANSWER

Detection: Generate a thread dump using jstack <pid> or tool like VisualVM/JConsole. Thread dump will explicitly list "Found 1 deadlock".

Prevention: Always acquire multiple locks in a consistent, strict global order across all code paths, or use ReentrantLock.tryLock(timeout).

Q34: What is ReentrantLock and why use it over synchronized?

ANSWER

An implementation of Lock offering advanced concurrency features:

Non-blocking lock attempts via tryLock().

Timed wait acquisitions.

Fair lock policy selection (prevents thread starvation).

Interruptible lock acquisition via lockInterruptibly().

Q35: What is Atomic Variable (AtomicInteger) and CAS mechanism?

ANSWER

Atomic classes use hardware-level CPU Compare-And-Swap (CAS) instructions instead of heavy lock synchronization. CAS atomically updates a memory location value only if it matches an expected current value, retrying in a non-blocking loop on failure.

Q36: What is a Semaphore in Java?

ANSWER

A concurrency synchronization object maintaining permits. Threads request permits using acquire() and return permits using release(). Used to limit concurrent access to constrained resources (e.g., controlling maximum database connections).

Q37: What is ForkJoinPool and how does work-stealing work?

ANSWER

A specialized execution framework designed for recursive divide-and-conquer parallel execution.

Work-Stealing: Every worker thread maintains its own double-ended queue (Deque). Idle threads "steal" unprocessed tasks from the tail of busy threads' queues to maximize CPU core utilization.

Q38: Explain CopyOnWriteArrayList.

ANSWER

Thread-safe list implementation. Every write operation (add, set, remove) creates a completely new underlying array copy. Read operations proceed without locks or synchronization. Ideal for read-heavy, write-rare scenarios (e.g., event listener arrays).

Q39: What is the purpose of Phaser in Java concurrency?

ANSWER

Introduced in Java 7, a flexible synchronization barrier similar to CyclicBarrier and CountDownLatch, but supporting dynamic registration/deregistration of participating threads across multiple execution phases.

Q40: What is Daemon Thread in Java?

ANSWER

Low-priority background thread that provides service support to user threads (e.g., Garbage Collector). The JVM exits automatically when all non-daemon (user) threads terminate, killing running daemon threads. Set using thread.setDaemon(true) prior to start().

Section 3: Collections Framework & Memory Management (Q41–Q60)

Q41: Explain internal working of HashMap in Java 8+.

ANSWER

HashMap stores key-value pairs in an array of buckets (Node<K,V>[]).

Insertion: Calculates hash: (h = key.hashCode()) ^ (h >>> 16). Index calculation: index = (n - 1) & hash.

Collision: Colliding elements form linked lists.

Treeification: If bucket linked list length exceeds threshold 8 and total capacity ≥ 64, the bucket converts from a linked list to a Red-Black Tree (O(N)→O(logN) performance optimization).

Q42: What is Load Factor and initial capacity in HashMap?

ANSWER

Initial Capacity: Default length of bucket array (16).

Load Factor: Measure of how full the map gets before resizing (Default 0.75).

Threshold: Capacity × Load Factor = 16 × 0.75 = 12. Resizing doubles capacity (to 32) and rehashes elements when total entry count exceeds 12.

Q43: Difference between fail-fast and fail-safe Iterators.

ANSWER

Fail-Fast (ArrayList, HashMap): Iterates over direct collection structures. Throws ConcurrentModificationException immediately if collection is modified structural-wise during iteration (tracks modCount).

Fail-Safe (CopyOnWriteArrayList, ConcurrentHashMap): Iterates over a cloned snapshot or view of the collection. Does not throw exceptions if modified during iteration.

Q44: Differentiate Comparable vs Comparator.

ANSWER

Comparable<T>: Defines natural sorting order inside the class by implementing compareTo(T o). Modifies class code directly.

Comparator<T>: Defines custom multiple sorting strategies externally via compare(T o1, T o2). Integrates cleanly with Lambda expressions (Comparator.comparing(...)).

Q45: How does IdentityHashMap differ from standard HashMap?

ANSWER

IdentityHashMap uses reference equality (==) instead of logical object equality (equals()) when comparing keys and values.

Q46: Explain internal implementation of HashSet.

ANSWER

HashSet internally delegates storage to a HashMap<E, Object> instance. Elements added to HashSet serve as keys in the internal map, while a dummy constant object (PRESENT) serves as every value.

Q47: What is the difference between ArrayDeque and LinkedList for stack/queue usage?

ANSWER

ArrayDeque uses resizable arrays, offering higher memory efficiency, cache locality, and faster performance than LinkedList, which creates extra object nodes for every element. ArrayDeque should be preferred for queue/stack implementation.

Q48: How does TreeSet / TreeMap keep elements sorted?

ANSWER

Uses self-balancing Red-Black Tree data structures guarantees O(logN) time complexity for search, insertion, and deletion operations while keeping elements sorted according to natural ordering or supplied Comparator.

Q49: Why should custom classes overridden as keys in HashMap be Immutable?

ANSWER

If a key object changes state after insertion, its calculated hashCode() changes. Subsequent map lookups (get()) calculate a different hash index and fail to retrieve the object, causing memory leaks and unreachable values.

Q50: How does WeakHashMap prevent memory leaks?

ANSWER

Uses weak references (WeakReference) for keys. When a key object is no longer strongly referenced elsewhere in the application, Garbage Collector reclaims the key, and WeakHashMap automatically clears the corresponding entry.

Q51: How do ArrayList growth dynamics work?

ANSWER

Initial default capacity is 10. When capacity limit is reached upon element addition, ArrayList increases its capacity by 50% (N+N/2) using Arrays.copyOf().

Q52: Explain difference between PriorityQueue and general Queue.

ANSWER

Standard queue processes items FIFO. PriorityQueue orders elements according to priority based on natural order or a Comparator, storing elements internally in a binary min-heap (O(logN) insertion/poll).

Q53: What is Garbage Collection Root (GC Root)?

ANSWER

An anchor object reachable from outside the heap:

Active thread local variables/stack parameters.

System Loaded static classes/variables.

JNI (Java Native Interface) global references.

Unreachable objects from GC roots are candidate targets for Garbage Collection.

Q54: Explain the Garbage Collection process: Mark-Sweep-Compact.

ANSWER

Mark: Scans heap to identify and mark all reachable live objects starting from GC Roots.

Sweep: Deletes unreachable dead objects to free memory.

Compact: Relocates surviving objects together to contiguous memory spaces to remove fragmentation.

Q55: Compare G1 GC vs. ZGC garbage collectors.

ANSWER

G1 GC: Region-based collector for large heaps (>4GB). Delivers predictable pause times by prioritizing regions with most garbage.

ZGC (Z Garbage Collector): Ultra-low latency collector (<1ms pause times) operating concurrently on multi-terabyte heaps without stopping application execution.

Q56: How do you capture and analyze a Heap Dump?

ANSWER

Capture via command-line jcmd <pid> GC.heap_dump /path/dump.hprof or JVM parameter -XX:+HeapDumpOnOutOfMemoryError. Analyze using Eclipse Memory Analyzer (MAT) or JProfiler to inspect memory leak paths via retain-heap reference trees.

Q57: What is safe point in Garbage Collection context?

ANSWER

Execution point in thread application execution where thread state is consistent, allowing the JVM to safely stop threads without memory corruption to perform internal GC sweeps.

Q58: What is LinkedHashMap and how does LRU Cache build on it?

ANSWER

Maintains a doubly-linked list running through entries to preserve insertion or access ordering. Overriding removeEldestEntry() allows converting LinkedHashMap into a Least Recently Used (LRU) Cache eviction structure.

Q59: Why cannot primitive types be stored in Java Collections?

ANSWER

Java Collections rely on Object references (Object[]). Primitives are not inherited from java.lang.Object. Java auto-boxes primitives to Wrapper classes (int → Integer), which adds memory overhead.

Q60: What is EnumMap and why use it over HashMap?

ANSWER

Specialized map for enum keys. Internally backed by a flat array, offering speed and memory performance comparable to direct array indexing without hash calculation overhead.

Section 4: Java 8+ Features & Functional Programming (Q61–Q80)

Q61: What is a Functional Interface?

ANSWER

An interface containing exactly one abstract method (SAM). It can contain multiple default or static methods. Annotated with @FunctionalInterface. Serves as target type for Lambda expressions.

Q62: Explain standard core Functional Interfaces in java.util.function.

ANSWER

Function<T, R>: Method R apply(T t) — Accepts T, returns R.

Predicate<T>: Method boolean test(T t) — Accepts T, returns boolean.

Consumer<T>: Method void accept(T t) — Accepts T, returns nothing.

Supplier<T>: Method T get() — Accepts nothing, returns T.

Q63: Differentiate map() vs flatMap() in Streams.

ANSWER

map(): 1-to-1 mapping. Applies transformation to each element and wraps outputs into a Stream<T>.

flatMap(): 1-to-N mapping. Transforms each element into a Stream, then flattens/merges all nested streams into a single consolidated output stream.

Q64: What are Default Methods in Interfaces and why were they introduced?

ANSWER

Interfaces method implementations marked with default keyword. Introduced in Java 8 to allow interface API evolution without breaking existing implementing classes (e.g., adding stream() method to Collection interface).

Q65: How does Java resolve the Diamond Problem in Multiple Interface Default Method Inheritance?

ANSWER

If a class implements two interfaces containing identical default method signatures, the class fails compilation unless it explicitly overrides the method and specifies which super-interface method to invoke using InterfaceName.super.methodName().

Q66: What is Method Reference and its types?

ANSWER

Syntactic shorthand for Lambda expressions calling an existing method (Class::method).

Static method: Math::max

Instance method on explicit object: System.out::println

Instance method on arbitrary object type: String::compareToIgnoreCase

Constructor reference: ArrayList::new

Q67: What is Optional class and how to prevent anti-patterns?

ANSWER

Container object wrapping non-null values or empty states to prevent NullPointerException.

Anti-pattern: Calling optional.get() directly without checking isPresent().

Best Practice: Use functional chains optional.map(...).orElseGet(() -> defaultValue). Never use Optional for constructor parameters or method attributes.

Q68: What is lazy evaluation in Java Streams?

ANSWER

Intermediate stream operations (filter, map, sorted) are executed only when a terminal operation (collect, forEach, count) is invoked on the stream, avoiding unnecessary processing passes over source data.

Q69: Explain Collectors.groupingBy() vs Collectors.partitioningBy().

ANSWER

groupingBy(): Groups elements by arbitrary classification criteria, producing Map<K, List<V>>.

partitioningBy(): Groups elements into a Map<Boolean, List<V>> based on a boolean Predicate.

Q70: Parallel Streams overhead and when to avoid them?

ANSWER

Parallel streams divide processing workload using the shared ForkJoinPool.commonPool().

Avoid when: Dataset sizes are small, task computation overhead is lightweight, operations are blocking/I/O bound, or order must be preserved (forEachOrdered).

Q71: What are Java 11 HTTP Client features?

ANSWER

Replaced HttpURLConnection. Native modern client supporting HTTP/1.1 and HTTP/2, synchronous/asynchronous request models, WebSockets, and fluent builder APIs.

Q72: What are Records introduced in Java 16?

ANSWER

Concise immutable data carrier class representation. Automatically generates private final fields, getters, equals(), hashCode(), and toString().

```
public record UserDto(String name, String email) {}
```

Q73: What are Sealed Classes introduced in Java 17?

ANSWER

Restricts which subclasses can extend or implement a class/interface using the permits keyword, controlling class hierarchy inheritance.

```
public sealed class Shape permits Circle, Square {}
```

Q74: What is Pattern Matching for instanceof (Java 16)?

ANSWER

Eliminates explicit casting boilerplate after type checks.

```
if (obj instanceof String s) {
    System.out.println(s.toLowerCase());
}
```

Q75: Explain Text Blocks in Java 15.

ANSWER

Multiline string literals avoiding escape sequences like \n or \", enclosed within triple quotes """...""".

Q76: What are Virtual Threads (Project Loom - Java 21)?

ANSWER

Lightweight threads managed by JVM rather than OS. Reduces memory footprint per thread, allowing millions of concurrent threads on minimal hardware without thread-pool exhaustion during blocking I/O calls.

Q77: What is the difference between peek() and map() in Streams?

ANSWER

peek(): Intermediate operation taking a Consumer. Mainly intended for debugging without altering element values or stream state.

map(): Intermediate operation taking a Function to transform elements into new forms.

Q78: Explain VarHandle introduced in Java 9.

ANSWER

Safe, high-performance replacement for sun.misc.Unsafe. Offers memory access handles to fields and array elements with atomic, volatile, and CAS memory ordering guarantees.

Q79: What is Collectors.teeing() in Java 12?

ANSWER

Collector that merges results of two downstream collectors using a bi-function.

```
double average = list.stream().collect(
    Collectors.teeing(
        Collectors.summingDouble(i -> i),
        Collectors.counting(),
        (sum, count) -> sum / count
    )
);
```

Q80: What is short-circuiting stream operation?

ANSWER

Operations that do not process an entire stream to yield a result (e.g., findFirst, findAny, anyMatch, allMatch, limit).

Section 5: Spring Boot, Microservices & SQL / Persistence (Q81–Q100)

Q81: What is Dependency Injection (DI) and Inversion of Control (IoC)?

ANSWER

IoC: Architectural principle shifting bean lifecycle and dependency creation responsibilities from custom application code to the Spring Framework Container.

DI: Design pattern implementing IoC where dependencies are injected into a class via Constructor, Setter, or Field. Constructor injection is preferred for immutability and testability.

Q82: Explain Spring Bean Scopes.

ANSWER

Singleton (Default): Single bean instance per Spring ApplicationContext.

Prototype: New instance created upon every request/injection.

Request: Single instance per HTTP request lifecycle (Web).

Session: Single instance per HTTP session (Web).

Application: Single instance per ServletContext.

Q83: How does Spring Boot Auto-Configuration work?

ANSWER

Enabled via @SpringBootApplication (includes @EnableAutoConfiguration). Spring scans classpaths for libraries using META-INF/spring.factories or AutoConfiguration.imports. It evaluates conditions (@ConditionalOnClass, @ConditionalOnMissingBean) and configures default beans automatically.

Q84: What is the difference between @Component, @Service, @Repository, and @Controller?

ANSWER

All are stereotype annotations registering classes as Spring Beans.

@Component: Generic stereotype.

@Service: Annotates business layer components.

@Repository: Annotates DAO persistence layer. Translates database persistence exceptions into Spring DataAccessException.

@Controller: Annotates presentation layer handling HTTP endpoints.

Q85: How does @Transactional work internally in Spring?

ANSWER

Spring creates dynamic CGLIB/JDK proxies wrapping target beans. The proxy interceptor opens a database connection transaction before method execution, invokes target execution, commits on success, and triggers rollback if an unchecked exception (RuntimeException or Error) is thrown.

Q86: What is Circular Dependency in Spring and how to resolve it?

ANSWER

Occurs when Bean A depends on Bean B, and Bean B depends on Bean A.

Resolution: Redesign components (break coupling), use @Lazy injection on one dependency, or use setter injection instead of constructor injection.

Q87: What is Spring Boot Actuator?

ANSWER

Module providing production-ready monitoring endpoints inspecting application health, metrics, thread dumps, environments, and HTTP traces (/actuator/health, /actuator/metrics).

Q88: How to secure REST endpoints using Spring Security?

ANSWER

Configure a SecurityFilterChain bean defining authorization request rules, authentication protocols (OAuth2/JWT, HTTP Basic), session management policy (Stateless for REST), and CORS/CSRF protections.

Q89: What is JPA First Level Cache vs. Second Level Cache in Hibernate?

ANSWER

1st Level Cache: Enabled by default. Bound to the EntityManager / Session level. Cache invalidates when transaction closes.

2nd Level Cache: Application-wide cache across sessions. Backed by providers like Ehcache or Hazelcast. Explicitly enabled via @Cacheable.

Q90: What is N+1 Select Problem in Hibernate and how to fix it?

ANSWER

Occurs when fetching an entity list (1 query) results in N lazy-loaded queries fetching related child entities individually (1+N queries).

Fix: Use JOIN FETCH in JPQL, Entity Graphs (@EntityGraph), or FetchMode JOIN.

Q91: What is Spring Cloud API Gateway?

ANSWER

Edge service routing client traffic requests to underlying microservices. Handles cross-cutting concerns like dynamic routing, rate limiting, security, request filtering, and circuit breaking.

Q92: Explain Circuit Breaker pattern using Resilience4j.

ANSWER

Prevents cascading system failures across microservices.

Closed: Normal operation. Requests flow to service.

Open: Failure threshold reached. Calls trip open; requests fail instantly or execute fallback logic without hitting downstream service.

Half-Open: Periodically tests downstream service health with sample requests. Resets to Closed if successful.

Q93: What is the Saga Design Pattern in Microservices?

ANSWER

Manages distributed transactions across independent microservice databases. Replaces distributed two-phase commit (2PC).

Choreography: Services publish domain events triggering adjacent operations directly.

Orchestration: Central Orchestrator coordinates transaction workflow execution and issues compensating transactions to revert state on failures.

Q94: Differentiate SQL Index types: B-Tree vs Hash Index.

ANSWER

B-Tree Index: Standard balanced-tree structure supporting equality checks and range-based queries (<, >, BETWEEN).

Hash Index: Direct hash lookup table optimized for point equality checks (=), but incapable of accelerating range searches.

Q95: What are Database Isolation Levels and Read Phenomena?

ANSWER

Read Uncommitted: Allows Dirty Reads.

Read Committed: Prevents Dirty Reads, allows Non-Repeatable Reads.

Repeatable Read: Prevents Dirty & Non-Repeatable Reads, allows Phantom Reads.

Serializable: Prevents all phenomena; enforces sequential transaction execution.

Q96: What is optimistic locking vs pessimistic locking in JPA?

ANSWER

Optimistic Locking (@Version): Assumes few concurrent conflicts. Validates object version fields during update commits, throwing OptimisticLockException on mismatch.

Pessimistic Locking (LockModeType.PESSIMISTIC_WRITE): Locks database records directly at query execution using native SELECT ... FOR UPDATE, blocking other transactions.

Q97: What is Spring Data JPA Entity Lifecycle?

ANSWER

Transient: New instance, unattached to EntityManager, no database identity.

Managed/Persistent: Associated with active context, monitored for state change updates (dirty checking).

Detached: Persistent context closed; entity retains database identity but isn't monitored.

Removed: Marked for deletion upon next transaction commit.

Q98: Explain Centralized Configuration in Microservices using Spring Cloud Config.

ANSWER

Provides server-side centralized management of application properties across environments (Dev, QA, Prod). Backed by Git repositories, configuration parameters update dynamically without service redeployments via /actuator/refresh.

Q99: What is Distributed Tracing (Zipkin/Sleuth/Micrometer Tracing)?

ANSWER

Tracks HTTP requests traversing distributed microservice boundaries by injecting unique Trace ID (entire request path) and Span ID (individual service hop) headers into call contexts.

Q100: How do you handle Database Schema Migrations in Java application deployments?

ANSWER

Use versioned database migration tools like Flyway or Liquibase. Migration scripts (.sql files) run automatically during Spring Boot application startup, applying pending schema changes sequentially while tracking history in a dedicated table (flyway_schema_history).

Section 6: OOP Fundamentals & Collections Gap-Fill (Q101–Q115)

Q101: What is the difference between a Constructor and a Method?

ANSWER

Constructor: Special block used to initialize a new object. Has the same name as the class, no return type (not even void), and is invoked automatically via the `new` keyword. Cannot be called directly like a regular method.

Method: A named block of reusable logic that can have any name, must declare a return type (including void), and is called explicitly on an object or class reference, any number of times.

Q102: Explain Polymorphism and its two types in Java.

ANSWER

Polymorphism means "one interface, many implementations" — the same method call behaves differently depending on the object/context.

Compile-Time (Static) Polymorphism: Achieved via Method Overloading — multiple methods with the same name but different parameter lists, resolved by the compiler based on argument types at compile time.

Runtime (Dynamic) Polymorphism: Achieved via Method Overriding — a subclass provides its own implementation of a superclass method, resolved at runtime based on the actual object type (via dynamic method dispatch/vtable lookup).

Q103: Explain Method Overloading with an example, and the rules governing it.

ANSWER

Overloading lets a class have multiple methods with the same name but different parameter lists (different number, type, or order of parameters). Return type alone is NOT sufficient to overload.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    double add(double a, double b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
```

Q104: Explain Method Overriding with an example, and the rules governing it.

ANSWER

Overriding lets a subclass redefine a method inherited from its superclass, with the same name, same parameters, and a covariant (or same) return type. The method must not have a more restrictive access modifier than the parent's, and it cannot throw broader checked exceptions (see Q6).

```
class Animal {
    void makeSound() { System.out.println("Some sound"); }
}
class Dog extends Animal {
    @Override
    void makeSound() { System.out.println("Bark"); }
}
```

Q105: What is Vector, and how does it differ from ArrayList?

ANSWER

Vector is a legacy (pre-Collections Framework) resizable array implementation, functionally similar to ArrayList, but with every method synchronized, making it thread-safe at the cost of performance.

Key difference from ArrayList: ArrayList is unsynchronized (faster, but not thread-safe) and grows by 50% when full; Vector is synchronized and grows by 100% (doubles) by default. In modern code, prefer ArrayList with explicit synchronization (Collections.synchronizedList) or CopyOnWriteArrayList instead of Vector.

Q106: What is LinkedHashSet, and how does it differ from HashSet and TreeSet?

ANSWER

LinkedHashSet maintains insertion order using an internal doubly-linked list while still providing HashSet's O(1) average lookup via an underlying hash table.

HashSet: No guaranteed order. TreeSet: Sorted order (natural or Comparator-based), backed by a Red-Black Tree. LinkedHashSet: Insertion order preserved, useful when you need both fast lookups AND predictable iteration order (e.g., de-duplicating a list while preserving its original sequence).

Q107: What is an Iterator, and what methods does it provide?

ANSWER

Iterator is a universal interface for traversing any Collection one element at a time, without exposing its internal structure.

Core methods: hasNext() — checks if more elements exist; next() — returns the next element and advances the cursor; remove() — safely removes the last element returned by next() during iteration (the only safe way to remove while iterating, avoiding ConcurrentModificationException).

Q108: What is ListIterator, and how does it differ from Iterator?

ANSWER

ListIterator extends Iterator but is available only for List implementations. It supports bidirectional traversal (hasPrevious()/previous() in addition to hasNext()/next()), lets you obtain the current index (nextIndex()/previousIndex()), and allows in-place modification of elements during iteration via set(E e), not just removal.

Q109: What are Immutable Collections in Java, and how do you create them?

ANSWER

An immutable collection cannot be structurally modified (no add/remove/set) after creation; attempting to do so throws UnsupportedOperationException.

Java 9+: List.of(...), Set.of(...), Map.of(...) create truly immutable collections directly.

Pre-Java 9: Collections.unmodifiableList(list) wraps an existing mutable list in a read-only view — note the underlying list can still be changed through the original reference, unlike List.of().

Benefits: thread-safety without synchronization, safety when exposing internal collections from an API (defensive design), and protection against accidental mutation bugs.

Q110: What is Hashtable, and how does it compare to HashMap and ConcurrentHashMap?

ANSWER

Hashtable is a legacy (pre-Collections Framework) key-value store, synchronized on every method (like Vector), and notably does NOT allow null keys or null values (throws NullPointerException if attempted).

HashMap: Not synchronized, allows one null key and multiple null values, better single-threaded performance.

ConcurrentHashMap: Thread-safe like Hashtable but with much better concurrent throughput (segment/bucket-level locking instead of locking the entire map on every operation — see Q23), and also disallows null keys/values. In modern code, Hashtable is effectively obsolete — use HashMap for single-threaded use or ConcurrentHashMap for concurrent use.

Q111: What is a Custom Exception, and how do you create one?

ANSWER

A custom exception is a user-defined class extending Exception (checked) or RuntimeException (unchecked), used to represent domain-specific error conditions with meaningful names instead of generic exceptions.

```
public class InsufficientBalanceException extends RuntimeException {
    public InsufficientBalanceException(String message) {
        super(message);
    }
}

// Usage:
if (balance < amount) {
    throw new InsufficientBalanceException("Balance too low for withdrawal of " + amount);
}
Extend RuntimeException when the caller typically can't/shouldn't be forced to handle it (e.g.,
```

Q112: What is a Lambda Expression, and what problem does it solve?

ANSWER

A Lambda Expression is a concise way to represent an anonymous function — implementing a Functional Interface's single abstract method inline, without the boilerplate of an anonymous inner class.

```
// Before Java 8 (anonymous inner class):
Runnable r1 = new Runnable() {
    public void run() { System.out.println("Running"); }
};

// With Lambda:
Runnable r2 = () -> System.out.println("Running");

Comparator<String> byLength = (a, b) -> a.length() - b.length();
It solves the verbosity problem of functional-style code in Java and is the foundation that make
```

Q113: Explain basic Exception Handling: try, catch, finally, and checked vs unchecked exceptions.

ANSWER

try: Wraps code that might throw an exception. catch: Handles a specific exception type if thrown. finally: Always executes (used for cleanup), regardless of whether an exception occurred (see Q14 for finalize distinction).

Checked Exceptions: Must be either caught or declared with `throws` (compiler-enforced), e.g., IOException, SQLException — represent recoverable conditions the caller should anticipate.

Unchecked Exceptions (RuntimeException and subclasses): Not compiler-enforced, e.g., NullPointerException, IllegalArgumentException — typically represent programming errors.

```
try {
    int result = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("Cannot divide by zero: " + e.getMessage());
} finally {
    System.out.println("Cleanup runs regardless");
}
```

Q114: What is Database Normalization, and what do 1NF, 2NF, and 3NF mean?

ANSWER

Normalization is the process of structuring relational tables to reduce data redundancy and avoid update/insert/delete anomalies.

1NF (First Normal Form): Each column holds atomic (indivisible) values, no repeating groups — e.g., no comma-separated values in a single cell.

2NF: Must be in 1NF, and every non-key column depends on the ENTIRE primary key (relevant only for composite keys) — eliminates partial dependency.

3NF: Must be in 2NF, and no non-key column depends on another non-key column (eliminates transitive dependency) — e.g., storing a customer's city derived from their zip code in the same table as other customer data would violate 3NF.

In practice, most production schemas target 3NF, then selectively denormalize for read performance where needed.

Q115: What are the ACID properties of a database transaction?

ANSWER

Atomicity: The transaction's operations either all succeed or all fail together — no partial updates (rollback on failure).

Consistency: A transaction moves the database from one valid state to another, never violating defined constraints/rules.

Isolation: Concurrent transactions don't interfere with each other's intermediate state (governed by isolation levels — see Q95).

Durability: Once committed, changes survive even a system crash (typically via write-ahead logging).

Interviewers often ask you to explain each with a one-line banking example: a fund transfer (debit + credit) must be Atomic (both happen or neither), Consistent (total balance across accounts stays correct), Isolated (a concurrent read doesn't see a half-completed transfer), and Durable (once confirmed, it's never lost).

Section 7: JPA & Hibernate (Q116–Q125)

Q116: What is ORM, and what problem does Hibernate/JPA solve?

ANSWER

ORM (Object-Relational Mapping) is a technique that maps Java objects to relational database tables automatically, letting developers work with objects instead of writing raw SQL for every CRUD operation.

JPA (Jakarta Persistence API) is the specification (a set of interfaces/annotations); Hibernate is the most common implementation of that specification. JPA/Hibernate solves the "impedance mismatch" between object-oriented code and relational tables, auto-generates SQL, manages object state/caching (see Q89), and handles relationships between entities.

Q117: What is an Entity in JPA, and how do you define one?

ANSWER

An Entity is a plain Java class mapped to a database table, annotated with @Entity. Each instance represents one row; each field (by default) maps to one column.

```
@Entity
@Table(name = "employees")
public class Employee {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "full_name", nullable = false)
    private String name;

    private Double salary;
    // getters/setters
}
```

Q118: Explain common Entity Mapping annotations: @Table, @Column, @Id, @GeneratedValue.

ANSWER

@Table(name = "..."): Specifies the table name if it differs from the class name (optional — defaults to class name).

@Column(name = "...", nullable = false, length = 100): Customizes column name, nullability, length, uniqueness, etc. (optional — defaults to field name).

@Id: Marks the primary key field.

@GeneratedValue(strategy = ...): Defines how the ID is generated — IDENTITY (DB auto-increment), SEQUENCE (DB sequence object), AUTO (JPA picks based on DB), or TABLE (a separate table tracks IDs, rarely used).

Q119: Explain JPA Entity Relationships: @OneToOne, @OneToMany, @ManyToOne, @ManyToMany.

ANSWER

@ManyToOne: Many entities reference one (e.g., many Employees belong to one Department) — the foreign key lives on the "many" side.

@OneToMany: The inverse side of @ManyToOne (one Department has many Employees) — typically mapped with `mappedBy` to avoid owning the relationship twice.

@OneToOne: One entity relates to exactly one other (e.g., Employee and EmployeeProfile).

@ManyToMany: Both sides can relate to multiple records (e.g., Students and Courses) — implemented via a hidden join table, or an explicit join entity if extra columns (like enrollment date) are needed.

```
@Entity
class Department {
    @OneToMany(mappedBy = "department")
    private List<Employee> employees;
}

@Entity
class Employee {
    @ManyToOne
    @JoinColumn(name = "department_id")
    private Department department;
}
```

Q120: What are Fetch Types (LAZY vs EAGER) in JPA, and why does the default matter?

ANSWER

FetchType.LAZY: The related entity/collection is loaded only when actually accessed (on first `.get()` call), not when the parent is loaded. Reduces unnecessary queries and memory usage.

FetchType.EAGER: The related entity/collection is loaded immediately along with the parent, in the same (or an immediately-following) query.

Defaults: @ManyToOne and @OneToOne default to EAGER; @OneToMany and @ManyToMany default to LAZY. Using EAGER carelessly on large collections is a common cause of performance problems and is closely related to the N+1 problem (Q90) — generally prefer LAZY and fetch explicitly (via JOIN FETCH or an Entity Graph) only what you need.

Q121: What are Cascade Types in JPA, and what do they control?

ANSWER

Cascade types define which operations on a parent entity automatically propagate to its related (child) entities, via the `cascade` attribute on a relationship annotation.

CascadeType.PERSIST: Saving the parent also saves new children. CascadeType.MERGE: Updating the parent also updates children. CascadeType.REMOVE: Deleting the parent also deletes children. CascadeType.ALL: Applies all cascade operations.

Example: if Department has `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)` on employees, deleting a Department also deletes all its Employees — use cascade carefully, since it can cause unintended mass deletions if misapplied on a shared/reference entity.

Q122: What is JPQL, and how does it differ from native SQL?

ANSWER

JPQL (Jakarta Persistence Query Language) is an object-oriented query language that operates on Entity objects and their fields, not database tables and columns directly — JPA translates it into the appropriate SQL for the underlying database at runtime (making it database-portable, unlike native SQL).

```
@Query("SELECT e FROM Employee e WHERE e.salary > :minSalary")
List<Employee> findHighEarners(@Param("minSalary") double minSalary);
Key difference from native SQL: JPQL references Entity class names and field names (e.g., `Emplo
```

Q123: How does Spring Data JPA simplify CRUD operations?

ANSWER

By extending JpaRepository<Entity, IdType>, Spring Data JPA auto-generates implementations for common CRUD operations (save, findById, findAll, deleteById, etc.) at runtime via a dynamic proxy — no implementation code needed.

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    List<Employee> findByDepartment(String department);  // Auto-implemented from method name
    Optional<Employee> findByEmail(String email);
}

// Usage in a service:
employeeRepository.save(employee);
employeeRepository.findById(1L);
employeeRepository.deleteById(1L);
Spring Data JPA also supports derived query methods — it parses method names like `findByDepartm
```

Q124: What is the difference between save(), saveAndFlush(), and flush() in Spring Data JPA?

ANSWER

save(): Persists/updates the entity in the persistence context, but the actual SQL may be deferred until the transaction commits or a flush occurs (due to Hibernate's write-behind caching).

flush(): Forces the persistence context to synchronize pending changes to the database immediately, without ending the transaction.

saveAndFlush(): Combines both — saves the entity and immediately flushes, useful when you need the generated ID or DB-side defaults (like a trigger-computed column) available right away within the same transaction.

Q125: What is the Persistence Context in JPA/Hibernate?

ANSWER

The Persistence Context (backed by the EntityManager/Hibernate Session) is a first-level cache and change-tracking area that manages the lifecycle of entity instances within a transaction (see Q97 for the full lifecycle: Transient/Managed/Detached/Removed).

While an entity is "Managed" within the persistence context, Hibernate automatically tracks field changes (dirty checking) and generates UPDATE statements at flush/commit time, even without an explicit save() call — this is why calling a setter on a managed entity inside a @Transactional method is often enough to persist a change.

Section 8: Spring Boot Practical (Q126–Q140)

Q126: What is the Spring Bean Lifecycle?

ANSWER

The sequence a Spring-managed bean goes through: 1) Instantiation (constructor called), 2) Dependency Injection (fields/setters populated), 3) Aware interfaces invoked if implemented (e.g., BeanNameAware), 4) @PostConstruct method (or afterPropertiesSet() from InitializingBean) called — bean is now ready for use, 5) Bean is used by the application, 6) @PreDestroy method (or destroy() from DisposableBean) called during container shutdown.

```
@Component
public class CacheManager {
    @PostConstruct
    public void init() { System.out.println("Cache warmed up"); }

    @PreDestroy
    public void cleanup() { System.out.println("Cache cleared"); }
}
```

Q127: What is Component Scanning in Spring Boot?

ANSWER

Component Scanning is the mechanism by which Spring automatically discovers and registers classes annotated with @Component (and its specializations @Service, @Repository, @Controller) as beans, without manual XML/Java config registration.

@SpringBootApplication implicitly includes @ComponentScan, scanning the package it's declared in and all sub-packages by default. To scan additional packages outside that hierarchy, use @ComponentScan(basePackages = "com.other.package") explicitly.

Q128: How do you build a basic REST API endpoint in Spring Boot?

ANSWER

```
@RestController
@RequestMapping("/api/employees")
public class EmployeeController {

    private final EmployeeService employeeService;

    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<EmployeeDto> getEmployee(@PathVariable Long id) {
        return ResponseEntity.ok(employeeService.findById(id));
    }

    @PostMapping
    public ResponseEntity<EmployeeDto> createEmployee(@Valid @RequestBody EmployeeDto dto) {
        return ResponseEntity.status(HttpStatus.CREATED).body(employeeService.create(dto));
    }
}
@RestController combines @Controller + @ResponseBody (return values are serialized directly to J
```

Q129: What is a DTO, and why use it instead of exposing Entities directly in a REST API?

ANSWER

DTO (Data Transfer Object) is a plain object used to shape the data sent between layers (typically API request/response), decoupled from the internal Entity/DB structure.

Reasons to use DTOs instead of exposing Entities directly: avoids leaking internal DB structure/sensitive fields to API consumers, prevents lazy-loading serialization errors (Jackson trying to serialize an uninitialized LAZY collection outside a transaction), allows the API shape to evolve independently from the DB schema, and lets you combine/reshape data from multiple entities into one response.

Q130: How do you implement request Validation in Spring Boot?

ANSWER

Use Bean Validation annotations (from jakarta.validation.constraints) on a DTO's fields, combined with @Valid on the controller parameter to trigger validation automatically.

```
public class EmployeeDto {
    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Invalid email format")
    private String email;

    @Min(value = 0, message = "Salary cannot be negative")
    private Double salary;
}

@PostMapping
public ResponseEntity<?> create(@Valid @RequestBody EmployeeDto dto) { ... }
If validation fails, Spring throws MethodArgumentNotValidException automatically (handled via @C
```

Q131: How do you implement centralized Exception Handling in Spring Boot?

ANSWER

Use @ControllerAdvice (a global interceptor for exceptions across all controllers) combined with @ExceptionHandler methods, instead of try-catch blocks scattered across every controller.

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex) {
        return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(new ErrorResponse(ex.getMessage()));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException ex) {
        String message = ex.getBindingResult().getFieldError().getDefaultMessage();
        return ResponseEntity.badRequest().body(new ErrorResponse(message));
    }
}
This centralizes error formatting into one consistent JSON error shape across the entire API.
```

Q132: Explain the Controller-Service-Repository layered architecture in Spring Boot.

ANSWER

Controller: Handles HTTP concerns only — request mapping, extracting parameters, returning responses. Should contain no business logic.

Service: Contains business logic, orchestrates calls to one or more repositories, and is where @Transactional boundaries typically live.

Repository: Handles data access only (via Spring Data JPA interfaces, see Q123), with no business logic.

This separation keeps each layer independently testable (you can unit test Service logic by mocking the Repository, without needing a real DB or HTTP context) and keeps responsibilities clear as the codebase grows.

Q133: How do you implement Logging in a Spring Boot application?

ANSWER

Spring Boot uses SLF4J as the logging facade with Logback as the default implementation. Inject a logger per class (by convention, named after the class) instead of using System.out.println.

```
private static final Logger log = LoggerFactory.getLogger(EmployeeService.class);

public Employee findById(Long id) {
    log.info("Fetching employee with id={}", id);
    return repository.findById(id)
        .orElseThrow(() -> {
            log.warn("Employee not found for id={}", id);
            return new ResourceNotFoundException("Employee not found");
        });
}
Use parameterized logging (`{}` placeholders) instead of string concatenation for performance, a
```

Q134: How do you write a basic unit test and integration test for a Spring Boot service?

ANSWER

Unit Test (fast, isolated): Use JUnit 5 + Mockito to mock dependencies, testing the Service's logic in isolation.

```
@ExtendWith(MockitoExtension.class)
class EmployeeServiceTest {
    @Mock private EmployeeRepository repository;
    @InjectMocks private EmployeeService service;

    @Test
    void shouldThrowWhenEmployeeNotFound() {
        when(repository.findById(1L)).thenReturn(Optional.empty());
        assertThrows(ResourceNotFoundException.class, () -> service.findById(1L));
    }
}
Integration Test (slower, loads real context): Use @SpringBootTest to load the full application
```

Q135: What is JWT, and how is it used for authentication in a Spring Boot REST API?

ANSWER

JWT (JSON Web Token) is a compact, self-contained, digitally signed token containing claims (user ID, roles, expiry) that the server can verify without a database lookup on every request — making it well-suited for stateless REST APIs.

Flow: 1) User logs in with credentials; 2) Server validates and issues a signed JWT; 3) Client sends the JWT in the `Authorization: Bearer <token>` header on subsequent requests; 4) A filter (e.g., in Spring Security) validates the token's signature and expiry on each request, extracting the user identity without needing a server-side session store.

Key properties: the payload is Base64-encoded (readable, not encrypted) but tamper-evident due to the signature — never put sensitive data like passwords inside a JWT payload.

Q136: How do you implement Pagination in a Spring Data JPA repository?

ANSWER

Extend PagingAndSortingRepository (or just use JpaRepository, which includes it) and accept a Pageable parameter.

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    Page<Employee> findByDepartment(String department, Pageable pageable);
}

// Controller usage:
@GetMapping
public Page<EmployeeDto> getEmployees(
        @RequestParam int page, @RequestParam int size) {
    Pageable pageable = PageRequest.of(page, size);
    return employeeRepository.findByDepartment("IT", pageable).map(this::toDto);
}
The returned Page<T> object includes the content plus metadata (total elements, total pages, whe
```

Q137: How do you implement Sorting in a Spring Data JPA query?

ANSWER

Pass a Sort object directly, or combine it into a Pageable (as shown in Q136), to control result ordering without writing it into the query itself.

```
// Standalone sort:
List<Employee> employees = repository.findAll(Sort.by(Sort.Direction.DESC, "salary"));

// Combined with pagination, multiple fields:
Sort sort = Sort.by("department").ascending().and(Sort.by("salary").descending());
Pageable pageable = PageRequest.of(0, 10, sort);
This keeps sorting dynamic and client-controllable (e.g., via a `?sortBy=salary&direction=desc`
```

Q138: Explain basic Git workflow commands relevant to a Java developer's daily work.

ANSWER

git clone <url>: Copy a remote repository locally. git status: Show changed/staged files. git add <file>: Stage changes. git commit -m "message": Save staged changes locally. git push: Upload local commits to remote. git pull: Fetch and merge remote changes into the current branch.

git branch <name> / git checkout -b <name>: Create (and switch to) a new branch, typically one per feature/bugfix. git merge <branch>: Combine another branch's changes into the current one. git rebase <branch>: Reapply current branch's commits on top of another branch's tip, producing a cleaner, linear history than merge (but rewrites commit history — avoid rebasing commits already pushed/shared).

Q139: What is the difference between git merge and git rebase, and when would you use each?

ANSWER

git merge: Creates a new "merge commit" combining two branch histories, preserving the exact history of when each branch diverged — non-destructive, safe for shared/public branches.

git rebase: Moves/replays your branch's commits onto the tip of another branch, producing a linear history with no merge commit — cleaner to read, but rewrites commit hashes, so it should only be used on local/private branches not yet pushed, or ones you're certain no one else is building on.

Common team convention: rebase your local feature branch onto main before opening a PR (to keep history clean and resolve conflicts early), but merge (not rebase) when integrating a PR into main.

Q140: What are good Refactoring practices for improving legacy or messy Java code?

ANSWER

Extract Method: Break a long method into smaller, well-named methods, each doing one thing — improves readability and testability.

Replace Magic Numbers/Strings with named constants: e.g., `MAX_RETRY_COUNT = 3` instead of a bare `3` scattered in code.

Reduce deep nesting: Use early returns/guard clauses instead of deeply nested if-else blocks.

Favor Composition over Inheritance (see Q17) when a class hierarchy becomes fragile.

Always refactor under test coverage: write or confirm tests exist BEFORE refactoring, so behavior changes are caught immediately — refactoring without tests is just "changing code and hoping."

Refactor in small, reviewable steps rather than one large rewrite, keeping the application deployable at every step.